

Ismeat: variable selection strategy for interval branch and bound

Journal of Global Optimization, 2017

Ignacio Araya Bertrand Neveu

El branching decide el costo del B&B

Contexto

La optimización global continua se resuelve deterministamente con *interval branch and bound*.

- Cada nodo se bisecciona en dos subproblemas.
- La variable elegida condiciona el tamaño del árbol.
- Se busca una solución ϵ -óptima sin explorar demasiados nodos.

Limitación

Las estrategias clásicas usan información local del nodo, pero ignoran qué restricciones importan más.

Las heurísticas clásicas pesan todo por igual

Reglas de referencia

- `round-robin`: rota variables sin usar información local.
- `largest-first`: elige el mayor dominio.
- `smear`: estima impacto con derivadas intervalares.

Dónde fallan

Idea

Si una restricción está activa, su efecto guía el branching; si está inactiva, aporta poco.

$$\text{smear}(x_i, g_j) = \text{wid}(x_i) |a_{ji}|$$

Mide ancho por sensibilidad local.

$$\text{smearsum}(x_i) = \sum_{j=1}^m \text{smear}(x_i, g_j)$$

Suma el efecto de todas las restricciones por igual.

Mensaje clave

El agregado puede ser engañoso: restricciones inactivas pueden dominar la elección.

round-robin, largest-first y smear

Diapositiva pendiente de revisión

La idea: usar multiplicadores de Lagrange óptimos

Definición

Se linealiza el problema alrededor del centro de la caja actual y se resuelve esa aproximación.

Idea

Los multiplicadores duales óptimos reponderan cada restricción antes de calcular el smear.

Selección

Se evalúa un Lagrangiano ponderado y se biseca la variable con mayor impacto.

Mensaje clave

No todas las restricciones pesan igual: la estrategia aprende su relevancia de la linealización.

El Lagrangiano explica qué restricciones importan

Definición

$$L(x, \lambda) = f(x) + \sum_{j=1}^m \lambda_j g_j(x), \quad \lambda_j \geq 0$$

- $f(x)$ es la función objetivo; la suma penaliza violaciones de restricción.
- Cada λ_j pondera la restricción $g_j(x) \leq 0$ asociada.
- λ^* son los **multiplicadores duales óptimos** del problema.
- λ_j^* mide la sensibilidad óptima frente a la restricción j .
- Si una restricción es inactiva, su multiplicador óptimo es 0.

lsmeas combina linealización y smear

Input: caja x , Jacobiano intervalar J , restricciones g

Output: variable a bisecar

```
1 linealizar  $g$  en  $\text{mid}(x)$ ;
2 resolver el LP con cotas de  $x$  y objetivo no acotado;
3 if el LP es infactible o no acotado then
4   return smearsum( $x, J$ );
5 for  $i = 1, \dots, n$  do
6    $D \leftarrow \lambda_i^*$ ;
7   for  $j = 1, \dots, m$  do
8      $D \leftarrow D + \lambda_{n+j}^* J_{ji}$ ;
9   medir  $|D \cdot \text{wid}(x_i)|$ ;
10 return arg max $_i |D \cdot \text{wid}(x_i)|$ ;
```

Idea central

La ponderación por multiplicadores duales óptimos aproxima la importancia real de cada restricción.

Implementación

IbexOpt integra lsmeas; el LP se resuelve con SoPlex.

Fallback

Si falla la fase lineal, la estrategia usa smearsum con el mismo J .

El experimento usa 76 instancias COCONUT

Configuración experimental

Benchmark	COCONUT, series 1 y 2; 76 instancias seleccionadas
Ejecuciones	Cinco por estrategia e instancia
Reporte	Se eliminan la mejor y la peor corrida; se promedian las tres centrales
Parámetros	Límite de tiempo 7200 s; precisión $\epsilon = 10^{-6}$
Plataforma	PowerEdge T420, dos Intel Xeon quad-core a 2.20 GHz y 8 GB RAM

Criterio de tiempo

Timeout significa que la estrategia no resolvió la instancia dentro de 7200 s.

lsmear domina a las estrategias clásicas

Métrica	lsmear	rr	lf	ssum	smax
Tiempo total	1071	1400	16	3035	2410
#box total	50555	661	12 626	1949	2552
Ganancia media vs. clásico	1.30–2.48	2.04	2.48	1.30	1.34

- lsmear obtiene el mejor tiempo total entre las estrategias comparadas.
- El **sobrecoste** del LP es pequeño frente al filtrado del solver.
- ssum es el mejor clásico, pero queda detrás en tiempo agregado.

lsmear-MG es la variante más simple y eficaz

Variante	Time total	Mejor time	Peor time	Mejor #box	Peor #box
lsmear	21035	0.6	2753	50	1877486
lsmear-MG	19181	0.6	1514	46	548282
lsmear-LR	20429	1.0	3865	11	203997
lsmear-LI	20429	1.0	3865	11	203997
lsmear-LRI	20429	1.0	3865	11	203997

- lsmear-MG logra el mejor tiempo total y el menor peor caso de #box.
- lsmear-LI y lsmear-LRI muestran el peor desempeño.
- lsmear-MG no requiere modificar el contractor del solver.

FeasibleDiving mejora el tiempo total

Instancia	minLB		FeasibleDiving		g
	ssum	lsmear-MG	ssum	lsmear-MG	
ex7_2_9	20	34	14	16	0.85
ex14_1_7	924	296	221	254	0.74
chem	7167	397	7173	381	17.1
himmel16	110	31	26	41	0.65
total time	59,274	29,136	43,867	19,181	2.29

- FeasibleDiving reduce el tiempo total frente a minLB.
- lsmear-MG obtiene el mejor total con FeasibleDiving.
- La ganancia media frente a `ssum` sube de 2.03 a 2.29.

La conclusión es un branching más informado

Idea principal

lsmeat pondera restricciones con multiplicadores duales óptimos y define un smear lagrangiano.

Mejor variante

lsmeat-MG ofrece el mejor compromiso entre rendimiento y simplicidad.

Ventaja estructural

Es independiente del contractor convexo usado por el solver.

Trabajo futuro

Incorporar errores de violación y tratar mejor variables lineales.